数据挖掘HW2报告

522031910213 朱涵

1 数据准备

下载数据集后，使用python进行数据清洗，包括进行合适的类型转换、过滤无效数据、添加衍生特征列等，其中选取了如下图的8个特征列保留，以NonStop列作为后续分类标签列。之所以选取这些特征列，是因为可以保留关键的价格、距离、时间等信息，并且其余列对于分类操作没有显著相关性，甚至可能因为其随机性而影响分析（比如Id列）。

```
self.selected_columns = [
    "flightDate",
    "startingAirport",
    "destinationAirport",
    "isNonStop",
    "baseFare",
    "totalFare",
    "travelDuration",
    "totalTravelDistance",
]
```

处理完成后结果如下图，其中由于内存限制，采用分块多线程处理，最终处理了约7600w行数据。

```
[2025-04-27 10:00:16] 所有分块已加载完成
[2025-04-27 10:00:16] 处理完成: 9272754 行
[2025-04-27 10:00:16] 处理完成: 9060919 行
[2025-04-27 10:00:16] 处理完成: 9346524 行
[2025-04-27 10:00:17] 处理完成: 9352049 行
[2025-04-27 10:00:17] 处理完成: 8907313 行
[2025-04-27 10:00:24] 处理完成: 9397265 行
[2025-04-27 10:01:38] 处理完成: 1988550 行
[2025-04-27 10:02:35] 处理完成: 9406065 行
[2025-04-27 10:03:19] 处理完成: 9312444 行
[2025-04-27 10:03:19]
处理摘要:
[2025-04-27 10:03:19] 总处理行数: 76043883
[2025-04-27 10:03:19] 总耗时: 1455.17 秒
[2025-04-27 10:03:19] 处理速度: 52257.79 行/秒
[2025-04-27 10:03:19] 峰值内存: 384.27 MB
[2025-04-27 10:03:19] 输出目录: flight_prices
```

查看csv文件，如下图。

flightDate	starti	destin	isNonSto	baseFare	totalFar	travelDu	totalT	year	month	day_o	fare_per_mile
10/25/2022	BOS	DFW	FALSE	133.95	167.6	7.53333	1570	2022	10	1	0.1067516
10/25/2022	BOS	DFW	FALSE	133.95	167.6	7.65	1672	2022	10	1	0.10023924
10/25/2022	BOS	DFW	FALSE	133.95	167.6	7.86667	1570	2022	10	1	0.1067516
10/25/2022	BOS	DFW	FALSE	133.95	167.6	8.26667	1570	2022	10	1	0.1067516
10/25/2022	BOS	DFW	FALSE	133.95	167.6	8.53333	1570	2022	10	1	0.1067516
10/25/2022	BOS	DFW	FALSE	133.95	167.6	8.86667	1570	2022	10	1	0.1067516
10/25/2022	BOS	DFW	FALSE	133.95	167.6	9.26667	1570	2022	10	1	0.1067516
10/25/2022	BOS	DFW	FALSE	133.95	167.6	9.53333	1570	2022	10	1	0.1067516
10/25/2022	BOS	DFW	TRUE	147.91	173.6	4.13333	1556	2022	10	1	0.11156813

2 频繁模式挖掘

2.1 原生python实现

使用python自行实现fp-growth算法，核心流程包括：

1. 构建FP树：扫描事务数据，统计项频次，过滤非频繁项后，按频次排序插入树中（共享前缀，计数累加）。
2. 递归挖掘：从树底部的低频项开始，回溯其前缀路径（条件模式基），构建子FP树，递归挖掘更短模式。
3. 输出结果：合并当前项与子树的频繁模式，生成完整项集（如 '[A, B]'）。

实现的代码如下图，包括3个关键函数。

```python
class FPTreeNode:
    def __init__(self, item, count, parent):
        self.item = item
        self.count = count
        self.parent = parent
        self.children = {}
        self.link = None

    def increment(self):
        self.count += 1


def construct_fp_tree(transactions, min_support):…

def mine_fp_tree(header_table, min_support, prefix, frequent_itemsets, frequent_items):…

def fp_growth(transactions, min_support):…
```

各个函数的作用为：

- create_fptree()：压缩数据为FP树；
- mine_fptree()：递归挖掘条件模式；
- find_prefixpath()：提取项的前缀路径。

2.2 SparkML库实现

比较简单，直接调用即可，代码如下：

```python
# 准备Spark DataFrame
df = spark.createDataFrame([(trans,) for trans in transactions], ["items"])

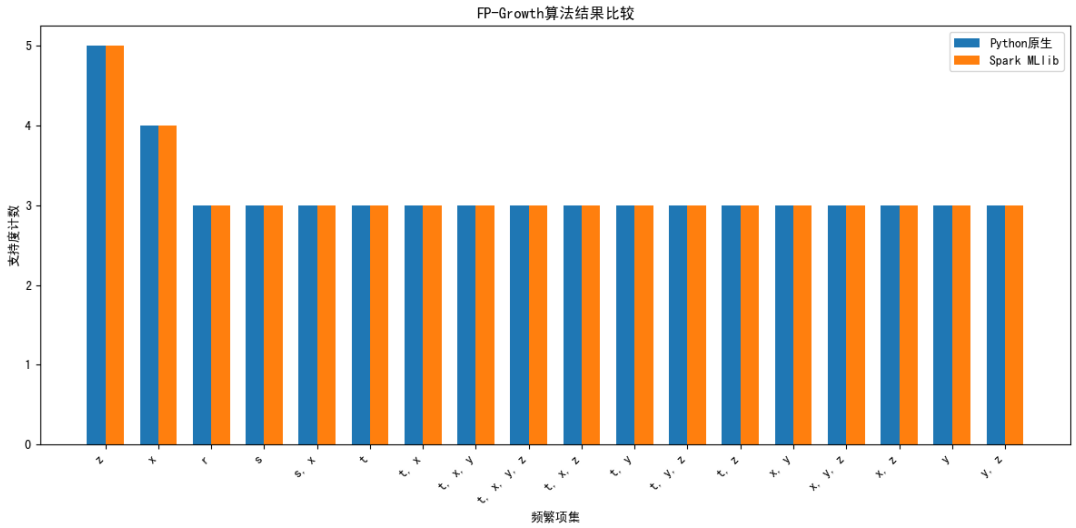
# 计算最小支持度比例
min_support_ratio = min_support / len(transactions)

spark_fp_growth = FPGrowth(
    itemsCol="items", minSupport=min_support_ratio, minConfidence=0.6
)
model = spark_fp_growth.fit(df)

spark_frequent_itemsets = model.freqItemsets.collect()
```

2.3 结果分析

结果如下图，包括各个频繁项的支持度计数。可以看到原生实现和ML库的结果是一致的，说明了原生实现的正确性。



3 决策树分类

3.1 原生实现

使用python自行实现决策树算法，核心流程包括：

1. 初始化：设置最大深度（`max_depth`）和最小分裂样本数（`min_samples_split`）。
2. 训练（`fit`）：递归构建决策树，计算每个节点的最佳分裂特征和阈值（基于Gini系数）。
3. 预测（`predict`）：遍历树结构，根据特征值判断样本归属类别。

实现的代码如下图，包括几个关键函数。

```python
class DecisionTree:
    """Python原生决策树实现"""

    def __init__(self, max_depth=5, min_samples_split=2):…

    def _gini(self, y):…

    def _best_split(self, X, y):…

    def _build_tree(self, X, y, depth=0):…

    def fit(self, X, y):…

    def _predict_sample(self, sample, tree):…

    def predict(self, X):…
```

各个函数的作用为：

- `_gini()`：计算数据集的Gini不纯度，衡量分类纯度。
- `_best_split()`：遍历所有特征和阈值，选择使加权Gini系数最小的分裂点。
- `_build_tree()`：递归构建树，终止条件包括深度限制、样本不足或纯度达标。
- `_predict_sample()`：单样本预测，递归查询树节点直至叶节点。

3.2 SparkML库实现

比较简单，直接调用即可，代码如下：

```
"""训练并预测"""
# 转换为Spark DataFrame
print("Converting data to Spark DataFrame...")
data = pd.DataFrame(X, columns=feature_cols)
data[label_col] = y
df = self.spark.createDataFrame(data)

# 特征向量化
print("Vectorizing features...")
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
df = assembler.transform(df)

# 划分训练测试集
print("Splitting data...")
train, test = df.randomSplit([0.7, 0.3], seed=42)

# 训练模型
print("Training Spark Decision Tree...")
dt = DecisionTreeClassifier(labelCol=label_col, featuresCol="features")
self.model = dt.fit(train)  # 保存模型

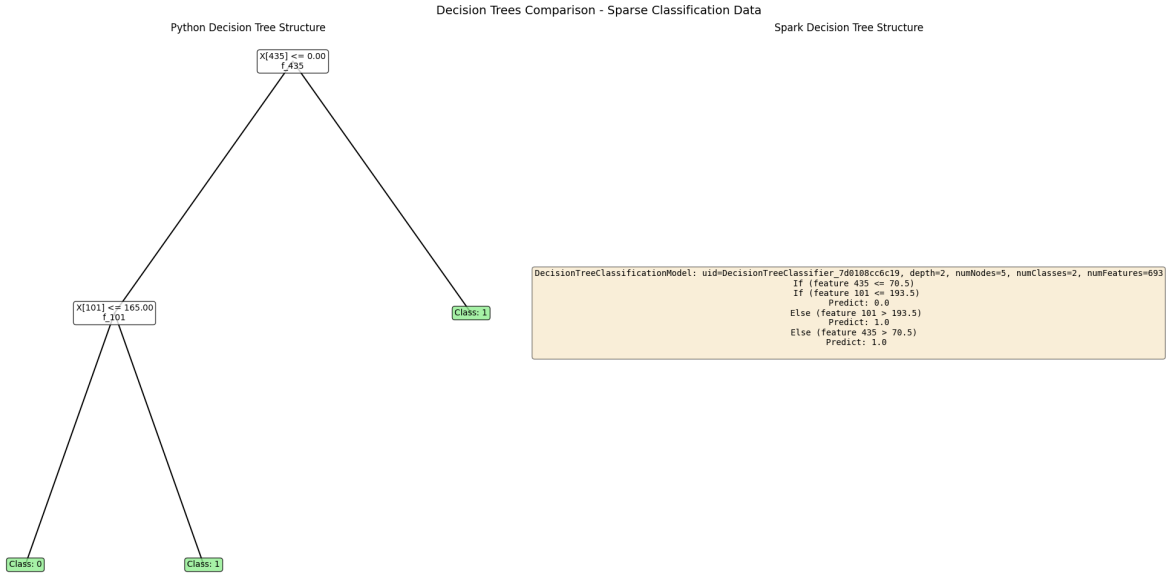
# 预测并收集结果
print("Making predictions...")
predictions = self.model.transform(test)
```

3.3 结果分析

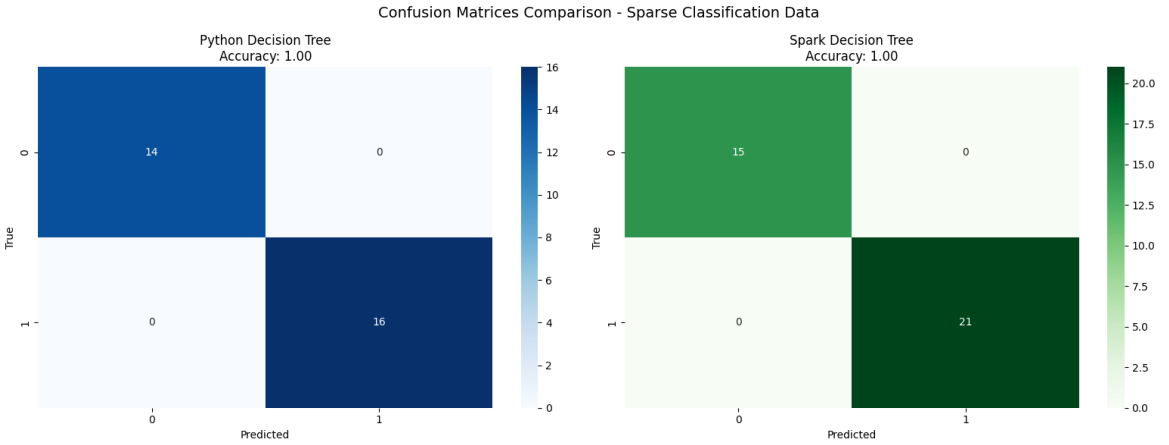
由于flight_prices完整数据集有近8kw行数据，在服务器硬件限制下，只能随机采样一部分数据进行分析。具体而言，完整数据集按时间顺序划分为了9块，在每一块内随机采样了10000条数据，最终聚合用于分析。数据集的划分皆为0.7的训练集与0.3的测试集，2种算法采用完全一样的划分方法。sample数据集采用所有特征作为候选进行决策，而flight_prices采用baseFare、totalTravelDistance、travelDuration三个特征作为候选进行决策。

3.3.1 sample数据集

2种算法生成的决策树如下图，可以看到2者的结构相似，都采用101和435特征列作为决策节点。



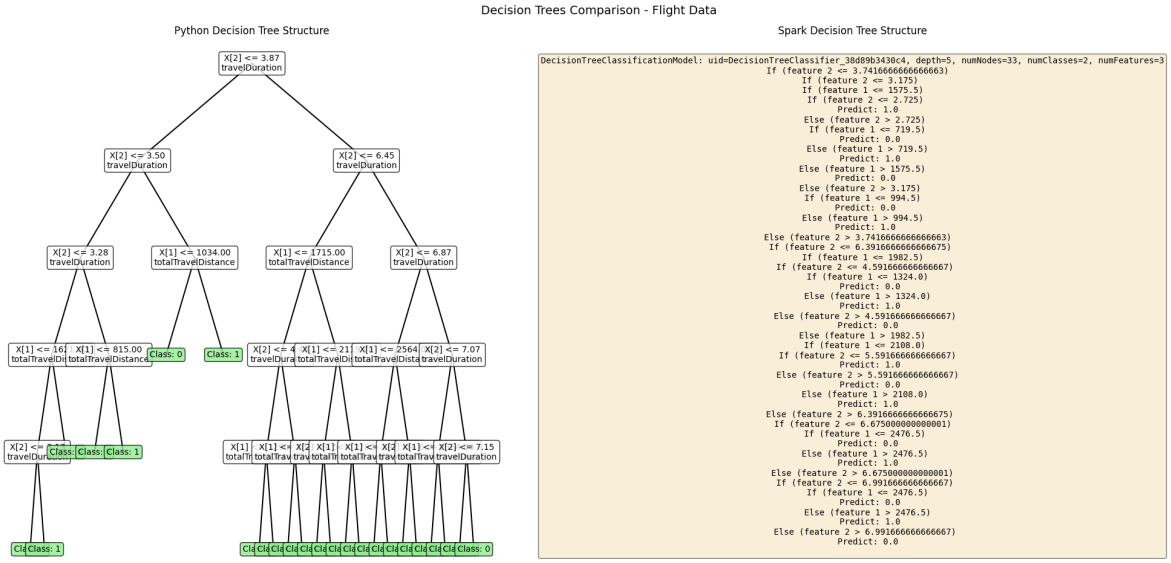
2种算法生成的决策树的预测准确率如下图：



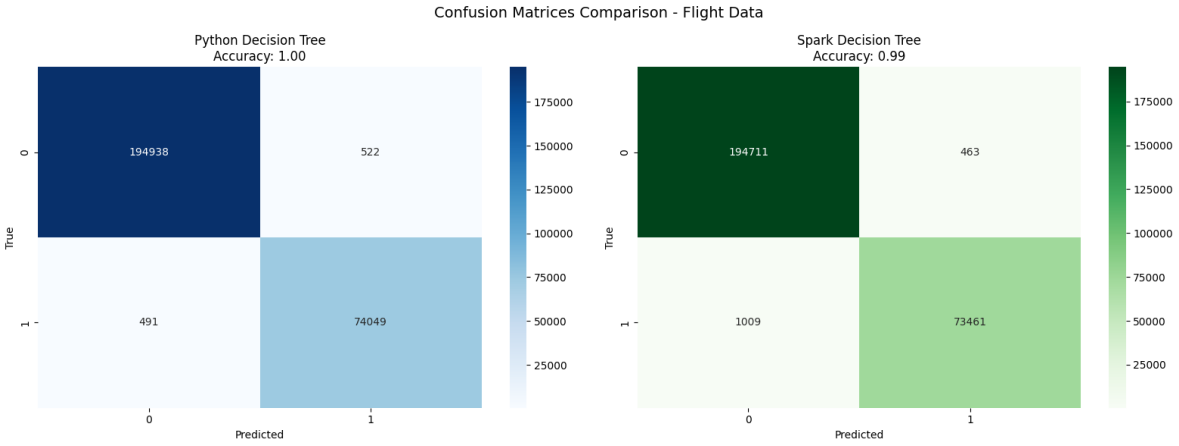
可以看到全部预测准确，由于数据量不大且比较简单，这样的准确率是合理的。

3.3.2 flight_prices数据集

2种算法生成的决策树如下图。可以看到2种算法都采用了travelDuration为首要分割特征，短时长倾向直飞，长时长多为中转；而原生实现的决策树决策最快只需要3层，即travelDuration $\leqslant$ 3.50 + totalTravelDistance $\leqslant$ 1034的行程是直飞；相比之下SparkML可能需要更复杂的逻辑。另外值得注意的是，2者的决策树都没有使用baseFare作为决策依据，推测是行程时长和距离已经足以进行决策，机票费用这个特征与前者线性相关，也就无需用于决策。



2种算法生成的决策树的预测准确率如下图，2者的准确率都接近1，原生实现的决策树准确率要略高于SparkML的，推测可能是随机数导致，亦或是SparkML在复杂的大数据集下表现会更好，而此数据集的特征过于简单，容易预测。



4 K-means聚类分析

由于硬件限制，在每个块内采样1000行数据，选取所有数值类型的特征进行聚类分析。

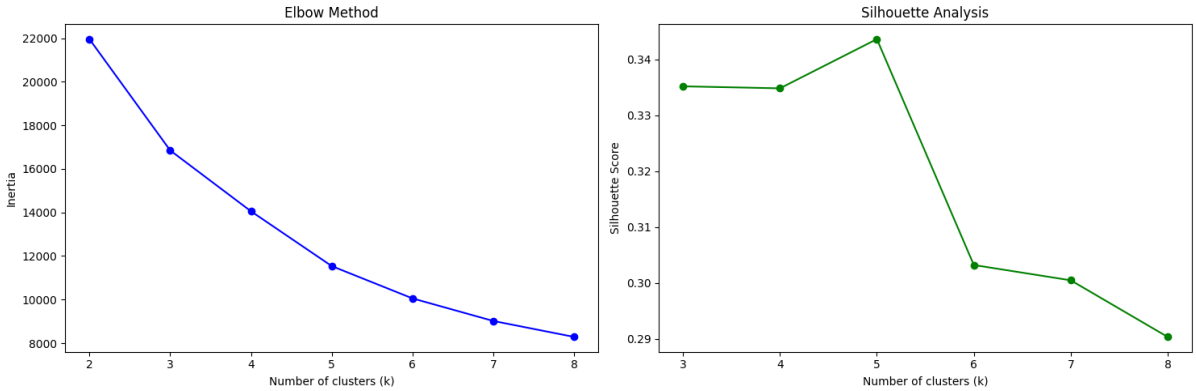
4.1 核心流程

1. 数据加载与预处理：从多个CSV文件采样合并数据，自动选择所有数值型特征并进行标准化处理，保存采样数据以便复用。
2. 确定最佳聚类数：通过肘部法则和轮廓系数评估K=2-8的聚类效果，自动选择轮廓分数最高的K值（如K=3）。
3. 聚类与可视化：用K-Means聚类后，通过PCA降维展示二维散点图，并绘制箱线图对比各簇特征分布，输出轮廓分数和标准化簇中心。

4. 结果保存：保存聚类标签数据及所有可视化图表（不同K值对比图、散点图、箱线图）到指定目录。

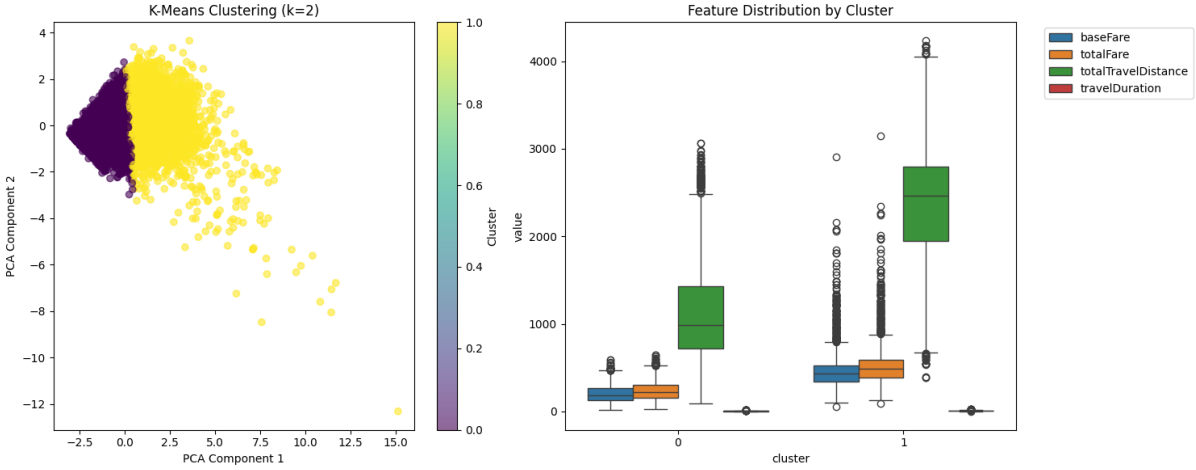
4.2 轮廓分评估

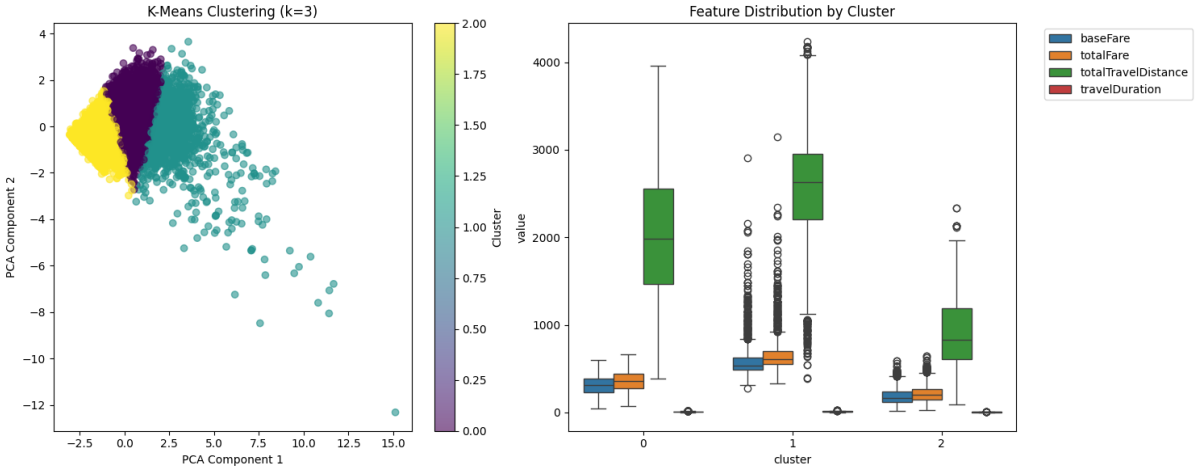
下面是肘部法则和轮廓系数评估的可视化结果，可以看到k=2时聚类效果最好，而k¿5后轮廓系数开始下降。由于原数据可以按IsNonStop特征进行分类，所以这个结果是符合预期的。

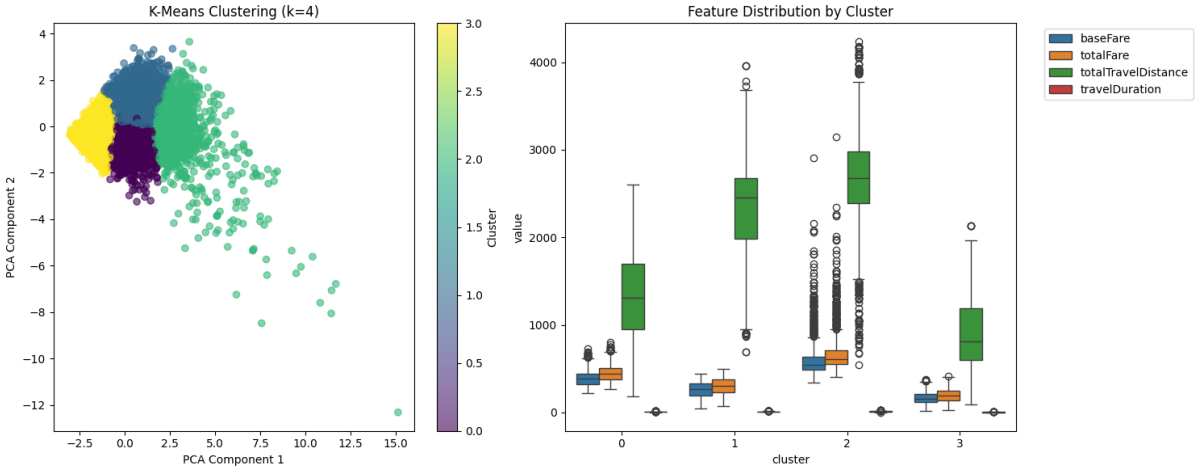


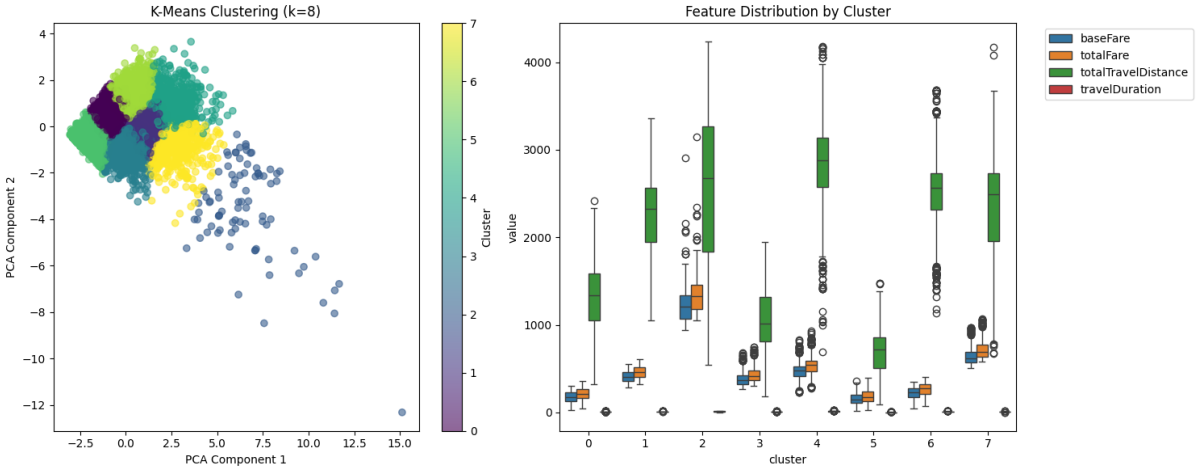
4.3 不同k的聚类结果

下面展示部分k的聚类结果，包括k=2,3,4,8的聚类可视化以及特征分布（其余结果可见result路径下图片）。其中为方便可视化，将特征先做PCA降维处理后再展示。可以看到k=2时聚类的效果是最好的。




K-Means Clustering (k=3)
PCA Component 1
PCA Component 2
Cluster
Feature Distribution by Cluster
cluster
value
baseFare
totalFare
totalTravelDistance
travelDuration


K-Means Clustering (k=4)
PCA Component 1
PCA Component 2
Cluster
Feature Distribution by Cluster
cluster
value
baseFare
totalFare
totalTravelDistance
travelDuration


K-Means Clustering (k=8)
PCA Component 1
PCA Component 2
Cluster
Feature Distribution by Cluster
cluster
value
baseFare
totalFare
totalTravelDistance
travelDuration